

Documento Técnico de Diseño de Base de Datos

Plataforma Híbrida Ecommify

1. Resumen Ejecutivo

Justificación de la sección: Esta sección se incluye para dar a los stakeholders una visión general rápida de la arquitectura seleccionada. **Contexto:** Se ha diseñado una arquitectura de base de datos híbrida para Ecommify que combina la solidez relacional de PostgreSQL (transaccional) con la flexibilidad documental de MongoDB (analítico y catálogo). Esta decisión permite optimizar cargas de trabajo mixtas (OLTP y OLAP), garantizando transacciones seguras para las compras, y escalabilidad dinámica para el catálogo de productos y el análisis de comportamiento.

2. Análisis de Requisitos

Cómo se hizo: Se analizaron los datasets originales de Olist (clientes, geolocalización, pedidos, pagos, productos) para identificar qué información requería integridad estricta y cuál requería flexibilidad. **Por qué se hizo:** Para aplicar el modelo adecuado a la naturaleza de los datos. No toda la información se comporta igual: los pagos no pueden fallar, mientras que los clics en productos varían rápidamente.

- **Requisitos Funcionales:** Procesamiento seguro de pedidos (ACID), cálculos rápidos de costo de envío basados en ubicación, y búsquedas de catálogo tolerantes a errores.
- **Requisitos No Funcionales:** Alta disponibilidad para el front-end de la tienda, particionamiento de datos históricos para no afectar el rendimiento transaccional.

3. Diseño Conceptual

Cómo se hizo: Se extrajeron las entidades principales del dataset de Olist y se relacionaron utilizando un modelo Entidad-Relación, implementado vía PlantUML. **Por qué se hizo:** El diagrama ER permite visualizar las reglas de negocio antes de la creación física en la base de datos, asegurando que las cardinalidades sean correctas (ej. un cliente puede tener múltiples pedidos, pero un pedido pertenece a un solo cliente).

(Referencia al diagrama: *docs/ER_Diagram_Ecommify.puml*) **Justificación de entidades clave:**

- **geolocation:** Separada para normalizar las ubicaciones por código postal y usar datos espaciales.
- **products:** Centro del modelo, enlazado transaccionalmente por `order_items`.

4. Diseño Lógico - Módulo PostgreSQL

Cómo se hizo: Se tradujo el modelo ER a scripts SQL DDL, aplicando tipos avanzados nativos de PostgreSQL y particionamiento. **Por qué se hizo:** PostgreSQL es excelente para garantizar la integridad referencial de los datos financieros (pedidos, pagos). El uso de características

avanzadas reduce la complejidad del backend y mejora la latencia.

Justificaciones de Tipos Avanzados y Extensiones:

1. **JSONB (specifications en products):** Se utilizó porque los atributos de los productos varían (un portátil tiene RAM, una camisa tiene talla). JSONB permite almacenar estos campos variables e indexarlos eficientemente sin crear tablas auxiliares gigantes (EAV anti-pattern).
2. **ARRAY (photos en products):** Una lista simple de URLs de imágenes se maneja mejor nativamente con un array unidimensional de texto, evitando una tabla intermedia `product_photos` y reduciendo el costo de los JOINS.
3. **Extensión PostGIS (geom en geolocation):** Habilita el cálculo preciso de distancias usando coordenadas esféricas, lo que optimiza la estimación de tiempos de envío y fletes.
4. **Extensión pg_trgm:** Se aplicó al nombre del producto para permitir búsquedas con errores ortográficos (fuzzy search) de manera rápida y sin motores externos (como Elasticsearch) en una etapa inicial.
5. **Particionamiento de orders:** Se aplicó particionamiento por rangos de fecha (`purchase_timestamp`). El objetivo es mantener el "hot data" (meses recientes) en índices pequeños y rápidos en la memoria RAM, mientras los pedidos antiguos se alojan en particiones archivadas.

5. Diseño Lógico - Módulo MongoDB

Cómo se hizo: Se modeló un esquema JSON centrado en documentos agregados, enfocado en las lecturas intensivas. **Por qué se hizo:** En un e-commerce, el catálogo se lee miles de veces más de lo que se compra. Si traemos el catálogo junto con todas sus reseñas desde PostgreSQL, requeriríamos costosos JOINS. MongoDB permite tener un documento `product_catalog` autónomo.

Justificaciones:

- **Embebido de reviews:** Se embebieron las reseñas directamente dentro del documento del producto porque siempre se consultan al mismo tiempo que el detalle del producto. Esto reduce las operaciones I/O en la base de datos.
- **Colección user_behavior_logs:** Creada para insertar clics y vistas sin saturar el motor transaccional. Es un esquema dinámico ideal para analítica de Big Data.

6. Decisiones Arquitectónicas Justificadas (Teorema CAP)

Cómo se hizo: Evaluamos la base de datos a la luz de Consistencia, Disponibilidad y Tolerancia a Particiones. **Por qué se hizo:** El teorema CAP dicta que ante una partición de red, debemos elegir entre estar disponibles o ser consistentes.

- **Para PostgreSQL:** Elegimos CP (Consistencia y Tolerancia a Particiones). Si hay un fallo de red o alta carga, preferimos que falle la compra a que se cobre un monto equivocado o se descuente un stock inexistente.
- **Para MongoDB:** Elegimos AP (Disponibilidad y Tolerancia a Particiones). Si el catálogo de productos está ligeramente desincronizado por unos milisegundos con respecto al inventario maestro, es un riesgo aceptable en pro de mantener la página viva y rápida para los usuarios.